

Problem A. Accidental Arithmetic

Source file name: Accidental.c, Accidental.cpp, Accidental.java, Accidental.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of January, 2025. You are working on your final report for the statistics class. However, when you enter the calculations into your trusted Brittle Arithmetic Portable Calculator, you notice that the $+$ and $-$ buttons have malfunctioned. Whenever you press a numeric button (a button from 0 through 9), an additional button press may be registered immediately after: either the $+$ button, or the $-$ button, or neither (but never both). “Ugh,” you think to yourself, “why did I not buy a Backup Arithmetic Portable Calculator...” But then you get an idea to save yourself from getting a low grade.

After some experimentation, you statistically determined that pressing a numeric button results in an additional $+$ input 45% of the time, and the same holds for the $-$ button (and never both at the same time). You decide that you should write your report about your findings. In addition to statistical analysis, the teacher requires you to perform some probabilistic predictions based on a statistical model. You decide to investigate what your calculator does when you try to simply input a natural number.



Your trusted Brittle Arithmetic
Portable Calculator.
Drawing by Freek Henstra

Given a natural number n , you consider entering the number into the calculator by pressing the numeric buttons corresponding to the standard base 10 representation of n . The calculator may register a $+$ or $-$ button press in between the digits of n , which results in an arithmetic expression. You wonder what happens if you press $=$ to evaluate the expression. Luckily, if your expression ends with a $+$ or $-$, the calculator ignores this, such that this evaluation always results in an integer. To finish your report, you decide to write a program that determines the expected value of this result for any given value of n .

Input

The input consists of:

- One line with an integer n ($0 \leq n < 10^{1000}$), the number that you will enter into the calculator.

Output

Output the expected value of the result.

Your answer should have an absolute or relative error of at most 10^{-6} .

Example

Input
12345
Output
5.4321
Input
777777
Output
42



Input
31415926535897932384626433832795028841971693993751
Output
141.5189

Problem B. Boggle Sort

Source file name: Boggle.c, Boggle.cpp, Boggle.java, Boggle.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of February, 2025. You have enjoyed another spirited evening of *Boggle* with your friends. After everybody left, you have thoroughly cleaned the apartment. All that is left is to bring the Boggle tray in order. You start to wonder: would it be possible to bring the Boggle tray in alphabetic order, without swapping any dice, but only by rotating them?

The Boggle tray consists of 16 six-sided dice. Each die is labelled with a letter from the English alphabet on each face. A single die contains a face labelled “Qu”. No letter appears 4 or more times on the same die. By turning a die once, you can move any of the sideways-facing letters up. Turning a die twice moves the downwards-facing letter up.



A tray of Boggle dice, out of order.
CC-BY-2.0 by Rich
Brooks on Wikimedia Commons

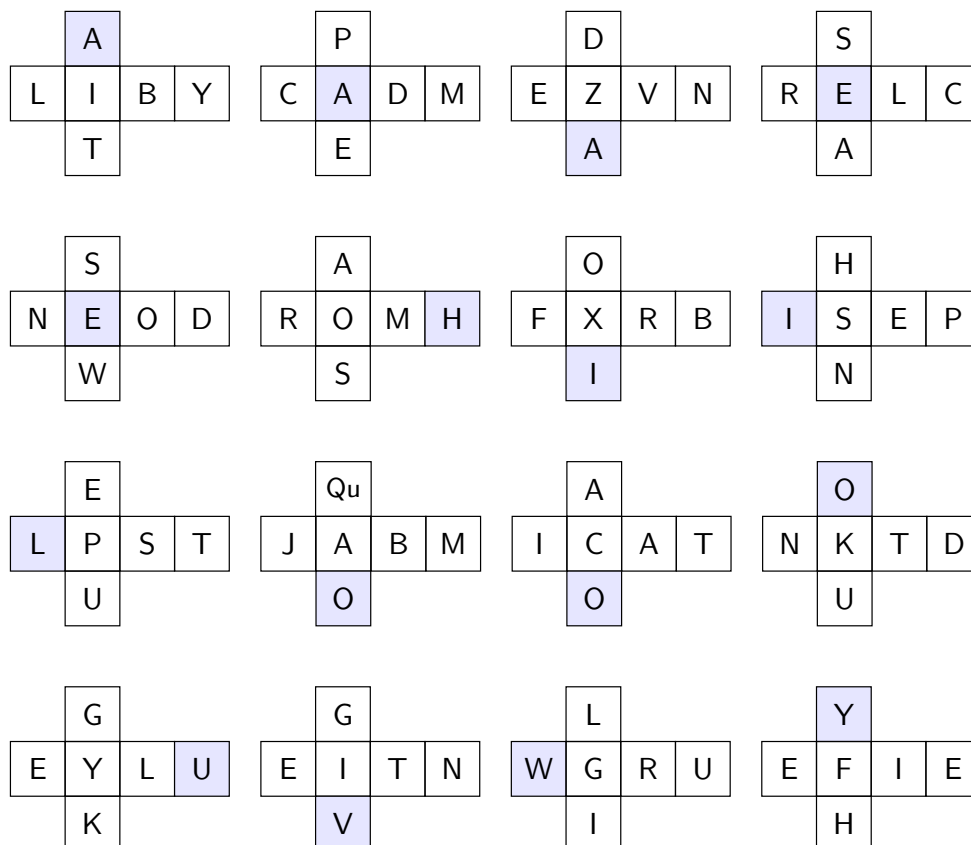


Figure 1: Visualization of the first sample input. The 16 Boggle dice are shown in reading order. For each die, the face in the center of the cross (“T” in the first die) is upwards-facing and the face on the far right (“Y” in the first die) is downwards-facing. The shaded die faces describe an optimal solution requiring 15 turns.

Bring the tray into alphabetically nondecreasing order, using standard reading directions (left-to-right,



top-to-bottom), using as few turns as possible. Letter case plays no role and the two-letter face is treated as “Q” followed by “U”, so “QuU” is sorted but “QuT” is not.

Input

The input consists of:

- One line with 16 letters, describing the currently upwards-facing faces of each die.
- Four lines with 16 letters, describing the currently sideways-facing faces of each die.
- One line with 16 letters, describing the downwards-facing faces of each die.

In each line, the i th letter describes the i th die for $1 \leq i \leq 16$.

All letters are English uppercase letters (A-Z).

The letter “Q” stands for the two-letter face “Qu” and appears exactly once in the input.

No letter appears 4 or more times on the same die.

Output

If it is possible to bring the tops of the dice into alphabetic order, output the minimum number of turns needed to do so. Otherwise, output “impossible”.

Example

Input	Output
IAZEEOXSPACKYIGF APDSSAOHEQAOGGLY LCERNRFILJINEEWE BDVLOMRESBATLTRI TEAAWSINUOUKVVIH YMNCDHBPTMTDUNUE	15
EXFETDMNMGDBSRM TIEGINOVRETACNUA PRYKASAEATNTSHID SOHUOEJDHVLYPLC UFIYAWBZONUIEIWE LBELCOQASIOLAEGP	impossible

Problem C. Coherency

Source file name: Coherency.c, Coherency.cpp, Coherency.java, Coherency.py
 Input: Standard
 Output: Standard
 Time / Memory limit: 5/14/14 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of March, 40025 CE in the world of *Battle Axe Player Clash 40,000* (BAPC40K). This futuristic table-top miniatures wargame is played with endearing figurines called *models*, each of which is placed on a circular *base*. The models are placed on a $100\text{ km} \times 100\text{ km}$ gaming board. A collection of such models forms a *coherent unit* if between any pair of models there is an unbroken chain of models that have a Euclidean distance of at most two inches¹ between the edges of their bases. Moreover, if the unit contains seven models or more, each model must be within two inches of at least *two* other models. Given the positions of a collection of models with varying base diameters, determine whether they form a single coherent unit.



The miniatures are typically hand-painted by the players.

One can prove that for any valid input for this problem, if the diameters of the circular bases differ from the given diameter by at most 10^{-5} mm, the coherency of a unit of models does not change.

Input

The input consists of:

- One line with an integer n ($2 \leq n \leq 2 \cdot 10^5$), the number of models.
- n lines, each with three integers x , y , and d ($0 \leq x, y \leq 10^8$, $d \in \{25, 28, 32, 40, 50, 65, 80, 90, 100, 130, 160\}$), describing a model that has its center coordinates at (x, y) and a base diameter of d , all given in millimeters.

Each model (including the base) fits on the gaming board.

It is guaranteed that no two models are overlapping, but the models can touch.

Output

If the n models form a single coherent unit, output “yes”. Otherwise, output “no”.

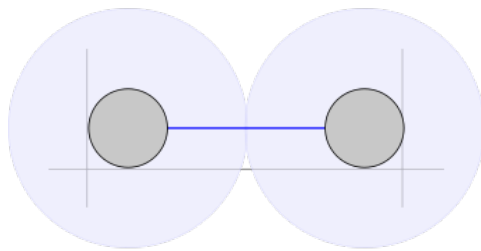
¹Recall that an inch equals 25.4 mm.



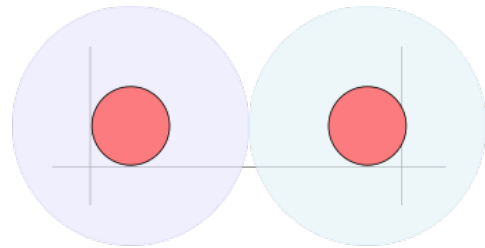
Example

Input	Output
2 13 13 25 88 13 25	yes
2 13 13 25 89 13 25	no
7 1255 1120 65 1204 1226 160 1090 1252 65 998 1179 160 998 1061 65 1090 988 160 1204 1014 65	no
7 1066 910 130 1007 1032 130 875 1062 130 770 978 130 770 843 130 875 758 130 1007 788 130	yes

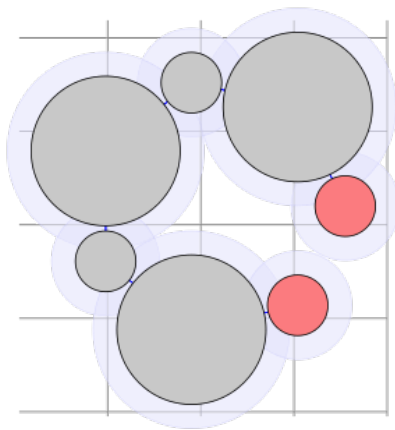
Explanation



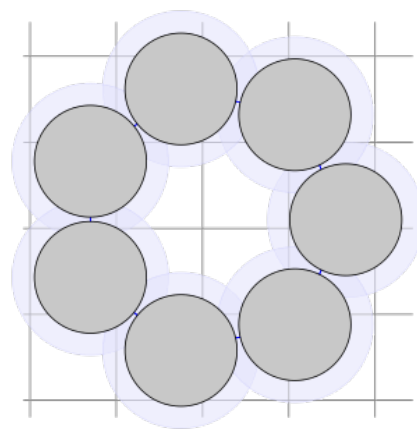
Sample 1



Sample 2



Sample 3



Sample 4

Figure 2: Illustration of the samples. Samples 1 and 4 are coherent. Sample 2 is not coherent, because the two models are too far away. Sample 3 is not coherent, because not all models are within two inches of two other models.

Problem D. Dart Game

Source file name: Dart.c, Dart.cpp, Dart.java, Dart.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

Consider an $n \times n$ grid where n is an odd number between 3 and 21, inclusive. Label the cells in the grid as follows:

(1,1)	(1,2)	(1,3)	...	(1,21)
(2,1)	(2,2)	(2,3)	...	
(3,1)	...			
...				
(21,1)	(21,2)	(21,3)	...	

The center of the grid is considered “bull’s eye”.

Given the grid size and where the dart has landed, determine the score. The picture below illustrates the scoring:

70	70	70	70	70	70	70
70	80	80	80	80	80	70
70	80	90	90	90	80	70
70	80	90	100	90	80	70
70	80	90	90	90	80	70
70	80	80	80	80	80	70
70	70	70	70	70	70	70

That is, if the dart is on the bull’s eye, score is 100; if the dart is on the square around the bull’s eye, score is 90; if the dart is two squares out, score is 80; if the dart is three squares out, score is 70; etc.

Input

There is only one input line; it contains three integers: n ($3 \leq n \leq 21$; also n is odd), indicating the grid size; r , a valid row number indicating where the dart has landed; and c , a valid column number indicating where the dart has landed.

Output

Print the score.

Example

Input	Output
5 2 1	80
13 5 12	50
21 11 11	100
21 1 19	0
21 20 4	10

Problem E. Excruciating Elevators

Source file name: Elevators.c, Elevators.cpp, Elevators.java, Elevators.py
Input: Standard
Output: Standard
Time / Memory limit: 2/2/2 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of May, 3025. The EEMCS building at TU Delft has grown to 10^6 floors beyond the ground floor! The floors are now numbered $0, 1, \dots, 10^6$, but there are still only four elevators. Moreover, the elevators have malfunctioned and are currently turned off. As employee of the Building Ascension Plans Company, you are tasked to fix the elevators.

You first have to order some components, which will take a month to arrive at the ground floor. After collecting the components, you must visit n floors $f_1, \dots, f_n$ in order. At each floor f_i , you must replace some component, which takes t_i seconds. After replacing the final component, the elevators will be fixed.

The stairs have long been removed, since people got tired of walking up millions of flights of stairs. You thus have to use the elevators to travel between the floors. You can turn on the elevators, but when you turn one on, you can no longer turn it off. Once turned on, an elevator will move up and down between floors 0 and 10^6 indefinitely. The elevators move at a speed of one floor every second without stopping, but you are agile enough to enter and exit.

You know exactly when the components will arrive. In the meantime, you can determine when to turn on each elevator. This timing determines the starting configuration of each elevator when the components arrive. Since you have a month, you can enforce any arbitrary starting configuration of the elevators. What is the minimum possible time to fix the elevators?

As an example, consider the first sample input. You can ensure one elevator starts at ground floor going up, and another starts at floor 750 000 going up. Entering the former elevator immediately, you arrive at floor 600 000 after 600 000 seconds. After 50 000 more seconds, you finish replacing the component at floor 600 000. In the meantime, the latter elevator reached the top floor after 250 000 seconds, when it started going down. Now, 400 000 seconds later, this elevator is at floor 600 000 going down. You can enter this elevator immediately and exit 200 000 seconds later at floor 400 000. Finally, after 150 000 seconds, you finish replacing the component at floor 400 000, and the elevators are fixed! The total time is $600\,000 + 50\,000 + 200\,000 + 150\,000 = 1\,000\,000$ seconds.

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 35$), the number of floors to visit.
- One line with n integers $f_1, \dots, f_n$ ($0 \leq f_i \leq 10^6$ for each i), the numbers of the floors you must visit.
- One line with n integers $t_1, \dots, t_n$ ($1 \leq t_i \leq 10^9$ for each i), the necessary time spent on each floor in seconds.

No two consecutive floors f_i, f_{i+1} are equal, and f_1 is non-zero.



The EEMCS building, now a million floors tall.



Output

Output the minimum possible time in seconds to fix the elevators after the components have arrived and you start at the ground floor.

Example

Input	Output
2 600000 400000 50000 150000	1000000
10 1 2 3 4 5 6 7 8 9 10 1 1 1 1 1 1 1 1 1 1	4000012

Problem F. Fences Make Good Neighbors

Source file name:	Fences.c, Fences.cpp, Fences.java, Fences.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/4/4 (C++/Java/Python) second(s) / 1024 megabytes

The King of Gletrian has a large estate that he wishes to divide up into triangular parcels which he then plans to give to deserving (that is, wealthy) followers. The estate is a convex polygon and already has fences around its borders, so the only cost now will be putting in new fences to perform the partitioning of the land. All fences will lie on straight lines between existing corners of the estate and no two fences will cross each other. Being fiscally minded (that is, cheap) he wants to put in the minimum amount of fencing possible.

But there is a problem (there always seems to be, doesn't there!). He has two sons who already have homes on the estate. One is a cute young lad and the other is a bit obtuse, but the problem lies in that they don't really get along with each other. Because of this, placing a single fence between their two parcels of land is out of the question – there will be constant bickering between them and at worst some type of physical altercation. However, the King has hope that his two sons might eventually learn to appreciate one another and he feels that all is needed is one good arbiter to serve as a liaison between them. To accomplish this the King wants to place exactly two fences between the brother's parcels, separating the brothers' lands by a single parcel where he'll place some voluntary (that is, conscripted) person to serve as go-between. Given these constraints, the King still wants to minimize the cost of the project, which means minimizing the length of fencing used. In addition, to avoid the brothers' houses, no potential fence which passes directly through a brother's location may be used in the triangulation.

An example (corresponding to Example Input 1) is shown in Figure 3, where the brother's locations are indicated by the two plus signs. The partitioning to the right, while using less fencing, is not a solution since there are more than two fences between the brothers' locations. A correct triangulation is shown on the left.

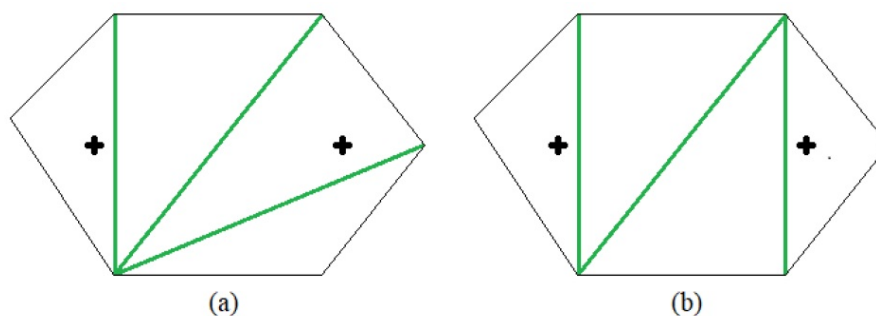


Figure 3: Example Input 1. (a) Correct solution. (b) Incorrect solution.

Input

Input starts with a single integer n ($6 \leq n \leq 500$), indicating the number of corners of the estate. Following this are n pairs of integers $x_i y_i$ ($|x_i|, |y_i| \leq 3000$) specifying the location of each corner, given in clockwise order. No two corner locations are the same and the polygon formed by connecting these corners is convex. No three consecutive corner locations are collinear. The last two lines each contain a pair of coordinates: the first of these lines contains $bx_1 by_1$ ($|bx_1|, |by_1| \leq 3000$), indicating the location of the first brother, and the second contains $bx_2 by_2$ ($|bx_2|, |by_2| \leq 3000$), indicating the location of the second brother. The two brother's locations are distinct and lie within the interior of the polygon. All coordinates are in kilometers.



Output

Output the minimum length of fencing in kilometers needed to satisfy all the constraints specified above. Answers within an absolute error of 10^{-3} of the judges' answer will be deemed correct. If it is not possible to satisfy the conditions stated above, output the -1 .

Example

Input	Output
6 0 -50 -40 10 0 50 80 50 120 0 80 -50 -10 0 100 0	354.553591
6 0 -50 -30 0 0 50 90 50 120 0 90 -50 0 5 90 -5	-1
12 0 100 50 86 86 50 100 0 86 -50 50 -86 0 -100 -50 -86 -86 -50 -100 0 -86 50 -50 85 60 60 -60 -60	1113.370332

Problem G. Geometry Rush

Source file name: Geometry.c, Geometry.cpp, Geometry.java, Geometry.py
Input: Standard
Output: Standard
Time / Memory limit: 1/2/2 (C++/Java/Python) second(s) / 1024 megabytes

You are playing the summer's hottest rhythm-based action platformer – Geometry Rush! The game is played on a 2D plane. Your character begins at $(0, 0)$ and every second must move at a 45-degree angle either up-right or down-right, which takes your character from position (x, y) to $(x + 1, y + 1)$ or $(x + 1, y - 1)$ respectively. You can change which direction you move every second, but not in between moves. There are obstacles protruding from the floor and ceiling that you must dodge. You win the game if, after w seconds, you reach the line $x = w$ without having touched any obstacles on the way.

The play area extends vertically from $y = -h$ to $y = h$. Obstacles are two polygonal curves: one curve starts at $(0, h)$ and ends at (w, h) and represents a ceiling of varying height. The x values of the vertices of this curve are non-decreasing, and the y values lie between $-h$ and h inclusive. A second polygonal curve starts at $(0, -h)$ and ends at $(w, -h)$ and represents the floor. Its vertices satisfy similar constraints.

Your character is a point of negligible extent: you can move from position (x, y) to $(x + 1, y \pm 1)$ so long as the line segment between your start and end position does not intersect either obstacle. (Exactly touching either polygonal curve counts as intersecting an obstacle, and loses the game.)

You have played *a lot* of games of Geometry Rush. To keep the game interesting, you have started to set challenges for yourself. For example: you win the game no matter where you cross the $x = w$ goal line. But for what maximum value of y can you win the game by crossing at (w, y) without touching any obstacles on the way? For what minimum value? Compute these numbers.

Input

The first line of the input contains four space-separated integers n , m , w , and h . The first two integers ($3 \leq n, m \leq 10^5$) are the number of vertices in the ceiling and floor polygonal curves, respectively. The second two integers ($3 \leq w, h \leq 10^5$) are the width and height of the play area, as described above.

The next n lines each contain two space-separated integers x and y ($0 \leq x \leq w$; $-h \leq y \leq h$): the coordinates of the vertices of the ceiling polygonal curve, in order from left to right. It is guaranteed that the first vertex is at $(0, h)$ and the last vertex is at (w, h) .

The next m lines each contain two space-separated integers x and y ($0 \leq x \leq w$; $-h \leq y \leq h$): the coordinates of the vertices of the floor polygonal curve, in order from left to right. It is guaranteed that the first vertex is at $(0, -h)$ and the last vertex is at $(w, -h)$.

For both polygonal curves: the x coordinates are non-decreasing, all vertices are distinct, and the curve does not self-intersect. Neither curve intersects $(0, 0)$. (The floor and ceiling curves might intersect each other, in which case the game is unwinnable.)

Output

If it is impossible to win the game, print **impossible**. Otherwise, print two space-separated integers: the minimum and maximum y values that the player could reach at $x = w$ without losing the game by touching an obstacle along the way.



Example

Input	Output
4 4 5 5 0 5 0 2 5 2 5 5 0 -5 0 -2 5 -2 5 -5	-1 1
4 4 6 5 0 5 0 2 6 2 6 5 0 -5 0 -2 6 -2 6 -5	0 0
3 3 7 5 0 5 5 -1 7 5 0 -5 2 1 7 -5	impossible
4 3 5 5 0 5 0 2 5 2 5 5 0 -5 3 -1 5 -5	-1 1



Problem H. Honest (or dishonest) Lottery

Source file name: Honest.c, Honest.cpp, Honest.java, Honest.py
Input: Standard
Output: Standard
Time / Memory limit: 1/2/2 (C++/Java/Python) second(s) / 1024 megabytes

You suspect your local lottery of cheating! Some numbers are coming up too often!

Each week, the lottery randomly selects five numbers in the range from one to fifty. So, each number should appear about 10% of the time if the numbers are truly chosen randomly. If a number appears far more than that, it's suspicious!

Determine if a set of lottery drawings is suspicious by listing all numbers that appear too often. To allow for random error, you'll need to flag any number that appears more than 20% of the time.

Input

The first line of input contains a single integer n ($1 \leq n \leq 10^3$). There will be $10 \cdot n$ lottery results for you to analyze.

Each of the next $10 \cdot n$ lines contains 5 integers x ($1 \leq x \leq 50$). Each line represents a drawing. All values on a line are unique.

Output

On a single line, output all numbers that appear strictly more than $2 \cdot n$ times in the list. If there is more than one, output them space-separated, in sorted order from smallest to largest. If there aren't any, output -1 .

Example

Input	Output
1 32 30 16 45 27 34 45 35 31 42 1 12 26 50 13 34 50 36 21 39 47 7 41 18 45 28 48 2 8 4 16 40 17 2 19 50 4 30 15 6 31 13 33 46 18 49 23 24 17 48	45 50

Problem I. Intermill Logistics

Source file name: Intermill.c, Intermill.cpp, Intermill.java, Intermill.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of September, 1825. Having just completed a record-setting wheat harvest, you wonder what to do with all this wheat. With a stroke of genius, you decide to use all of this wheat to bake your favourite type of cookie: stroopwafels. Of course, all this wheat should be ground to flour first. Because you cannot wait to start baking, you want to do this as fast as possible, so you decide to contact all flour mills in the Netherlands to ask them for help.

For each of these flour mills, you know how fast they can grind wheat to flour and how long it takes for a shipment to travel to and from the mill. You have enough grain ships available that can transport wheat to these mills, and bring the flour back from the mills. Dividing the wheat optimally between these mills, how long does it take until you have received back all the wheat?

As an example, consider the first sample case. To divide the wheat optimally between the three mills, you ship 400 kilograms to the first, 120 to the second, and 480 to the third. The first mill requires 5 hours to grind its wheat, the second requires 1 hour, and the third requires 3 hours. Combined with the shipping time to and from each mill, you receive all wheat back after exactly 11 hours.

In the second sample case, we send all of the wheat to the first mill. This mill can grind all 100 kilograms of wheat in 1 hour, which together with the 2 hours for shipping back and forth results in a total of 3 hours. As the shipping for the second mill would already take 4 hours, it is optimal to only use the first mill.



One of the flour mills.

Input

The input consists of:

- One line with two integers n and w ($1 \leq n \leq 10^5$, $1 \leq w \leq 10^9$), the number of mills and the amount of wheat you have, in kilograms.
- n lines, each with two integers p and t ($1 \leq p, t \leq 10^9$), describing a mill that can process p kilograms of wheat per hour, located t hours away.

Output

Output the number of hours until you have received back all the wheat, when dividing the wheat optimally between the mills.

Your answer should have an absolute or relative error of at most 10^{-6} .

**Example**

Input	Output
3 1000 80 3 120 5 160 4	11
2 100 100 1 500 2	3
3 7 1 1 1 1 1 1	4.3333333



Problem J. Jump ... Triple Jump

Source file name: Jump.c, Jump.cpp, Jump.java, Jump.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

You're watching your friend playing a platformer game. In this game, when a player jumps in a straight line, there are three different integer distances they can choose to travel. You don't know what the three are, but hope to find out by watching the gameplay.

You've seen your friend perform many "triple jumps" (i.e. three jumps in a row in the same direction) and recorded the total distance they traveled over all three jumps combined. In each triple jump, your friend can use any of the three integer differences, including repeating the same distance one or more times. After a long time of watching, you've recorded a list of unique distances, and are sure that all possible triple jump distances are in this list.

Given the distances you wrote down, can you recover the three distances?

Input

The first line of input contains a single integer n ($7 \leq n \leq 10$), which is the number of unique observations you made.

The next line contains n space separated integers d ($1 \leq d \leq 10^3$). These are your observations, in strictly ascending order.

Output

Output three space-separated integers on a single line. These are the three jump distances. Output them in sorted order, smallest to largest. The three jump distances are guaranteed to be unique.

Example

Input	Output
10 6 9 10 12 13 14 15 16 17 18	2 5 6

Problem K. Knowing the Clock

Source file name: Knowing.c, Knowing.cpp, Knowing.java, Knowing.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of November, 1625. You have put a lot of time into making a mechanical watch, but you are unsure if the hands are placed correctly. Even a watch that is not running is correct twice a day, but if the hands of the watch do not correspond to a real time, it is never correct at all. Without wasting any more time, you measure the angles of the hands and check whether they correspond to a real time.

For example, consider the first sample input, visualized in Figure 4: if the minute hand points to a quarter past (90 degrees from 12 o'clock), then the hour hand cannot point exactly to 2 o'clock (60 degrees from 12 o'clock).

You know that both hands of the watch are moving continuously.



CC BY-SA 4.0 by Jordiferrer on Wikimedia Commons

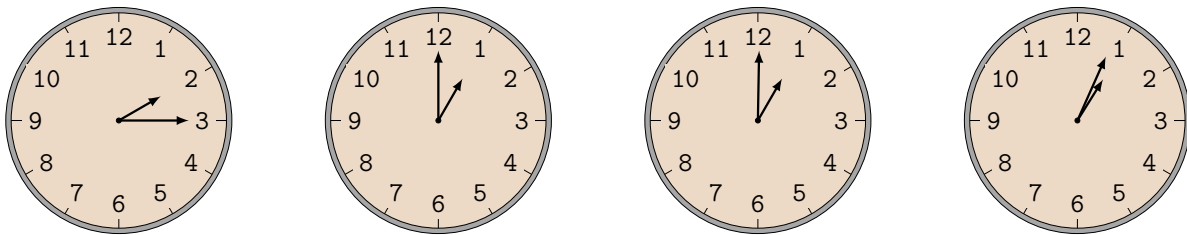


Figure 4: Illustrations of the sample inputs. The first and third sample inputs *do not* correspond to a real time. The second and fourth sample inputs *do* correspond to a real time.

Input

The input consists of:

- One line with two integers h and m ($0 \leq h, m \leq 359$), the exact clockwise angle that the hour hand is from 12 o'clock and the exact clockwise angle that the minute hand is from 12 o'clock. Both angles are given in degrees.

Output

If the angles of the hands correspond to a real time, output “yes”. Otherwise, output “no”.

Example

Input	Output
60 90	no
30 0	yes
30 1	no
32 24	yes

Problem L. Linguistic Labyrinth

Source file name:	Labyrinth.c, Labyrinth.cpp, Labyrinth.java, Labyrinth.py
Input:	Standard
Output:	Standard
Time / Memory limit:	10/10/10 (C++/Java/Python) second(s) / 1024 megabytes

It is the 25th of December, 2025. As a Christmas tradition, you gather a group of friends to solve a puzzle. Among your friends are *wordcels* and *shape rotators*, who are respectively better at thinking with words and with mental images. This puzzle challenges even the smartest wordcel and the most brilliant shape rotator:

There is a 3-dimensional grid with points at all integer coordinates (x, y, z) with $1 \leq x, y, z \leq n$, and each point has a label associated with it, which is either 'B', 'A', 'P', or 'C'. In this grid, you need to find occurrences of the curly word "BAPC". A curly word "BAPC" is a collection of four points in the grid such that:

- The labels spell out "BAPC" (in this order).
- The angle that the triplet "BAP" makes is 90 degrees: the vectors from $B \rightarrow A$ and from $A \rightarrow P$ form a 90-degree angle.
- The angle that the triplet "APC" makes is 90 degrees: the vectors from $A \rightarrow P$ and from $P \rightarrow C$ form a 90-degree angle.

Note that the two angles do not need to be axis-aligned. As an example, see the third example case, visualized in Figure 5.

How many occurrences of the curly word "BAPC" are in the given grid?

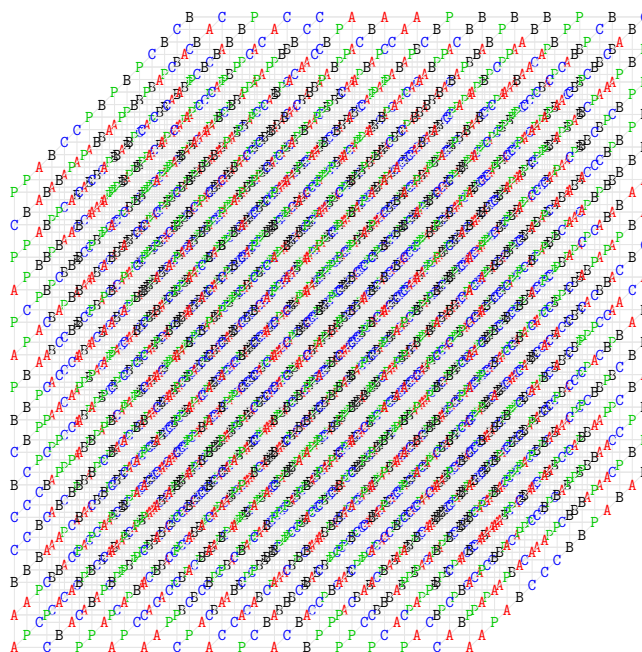
Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 22$), the size of the grid.
- n blocks of $n + 1$ lines. Each block of $n + 1$ lines consists of:
 - One line with a hyphen (-), to make the input more human-readable.
 - n lines with n characters, each character being either 'B', 'A', 'P', or 'C', representing all labels of one horizontal layer of the 3-dimensional grid.

Output

Output the number of curly words "BAPC" in the 3-dimensional grid.



Spoiler alert: this is one of the secret test cases.

Example

Input	Output
1 - B	0
2 - PA PB - CC PB	2
3 - BBB BCB BCB - BBC CBA BBB - BBB BPB BBB	2

Explanation

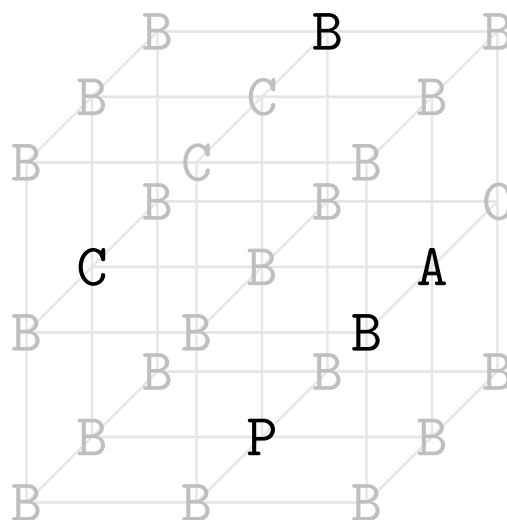


Figure 5: Visualization of the third example input. In this grid, there are two curly words “BAPC”, using the highlighted letters.

Problem M. Marching Orders

Source file name: Marching.c, Marching.cpp, Marching.java, Marching.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 1024 megabytes

Dean Bob Roberts is in charge of the order in which the professors of his college march in the graduation ceremonies. Because of complaints among certain professors from the newly created DEI Studies Department, it has been decided that the order in which they march should not be based on seniority but should be random. Bob thinks this is fine, and to be totally transparent he has communicated the following method he will use to create the marching list: He starts with an alphabetically ordered list of n professors numbered $0, 1, \dots, n - 1$ and a non-negative integer $m < 10^9$. Then the first person in the marching list is the one in position $m \bmod n$ in the alphabetic list. This shortens the alphabetic list by one, shifting down all those in positions greater than $m \bmod n$. The second person in the marching list is the one in position $m \bmod (n - 1)$, and so on.

For example, assume we have 6 professors A, B, C, D, E and F . If $m = 11679$, then the marching list is created as follows:

n	$m \bmod n$	alphabetic list	marching list
6	3	A B C D E F	D
5	4	A B C E F	D F
4	3	A B C E	D F E
3	0	A B C	D F E A
2	1	B C	D F E A C
1	0	B	D F E A C B

This sounds fair, but is not as transparent as some professors would like as Dean Roberts does not make public the value of m that he uses. This makes it difficult to determine if he is actually following the method he has stated or has just selected the marching order based on his personal whims and biases. What the faculty would like to know is, for a given marching order, is there a value of m which would produce that order?

Input

The first line of input contains a single decimal integer n ($5 \leq n \leq 20$), the number of professors who will be marching. The second line consists of a string containing a permutation of the first n uppercase letters of the alphabet, giving the proposed marching order.

Output

If the given marching order could not have come from the algorithm, output a single line containing the word NO. Otherwise, output two lines, the first containing the word YES and the second containing the smallest non-negative value of m which could produce the given marching order.

Example

Input	Output
6 DFEACB	YES 39
7 DFEGACB	NO